Vectors

Data Structures

- *vectors*
- factors
- lists
- matrices
- arrays
- dataframes

Vectors

Vectors are a one-dimensional data structure and are the simplest one in R.

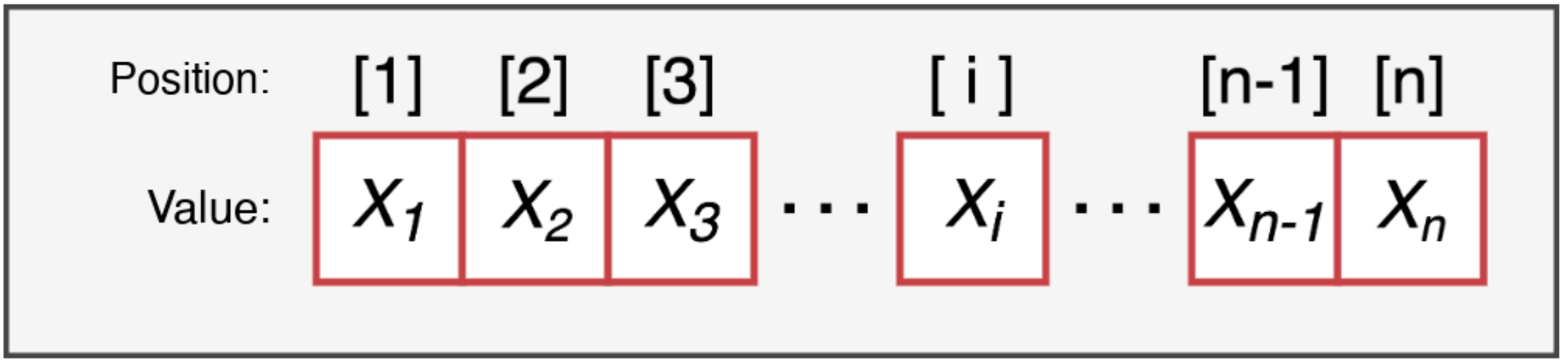
```
[1]  0  1  2 10
[1] "bob"   "alan"  "daisy" "paul"
```

Properties of a vector

- **length**: the number of elements the vector is made of
- **type**: the type of data the vector is made of. A vector must in fact be made of **elements all of the same type**!

Properties of the elements of a vector

- **a value**: the value of the element, which can be of any type, for example a number or a string of characters
- **a position index**: a positive integer that identifies its position within the vector.



Creating a vector

Vectors can be created with the `c()` command, writing inside the brackets the values of the elements in the desired order, separated by commas.

```
num_vect <- c(1,2,3,4)
# you can also use the seq() function
num_vect_seq <- seq(from = 1,to = 4, by = 1)
num_vect
```

```
[1] 1 2 3 4
```

```
num_vect_seq
```

```
[1] 1 2 3 4
```

Creating a vector

Vectors can be created with the `c()` command, writing inside the brackets the values of the elements in the desired order, separated by commas.

```r
char_vect <- c("R","R","R","ok")
# you can also use the rep() function
char_vect_rep <- c(rep("R", times = 3), "ok")
char_vect
```

```
[1] "R"  "R"  "R"  "ok"
```

```r
char_vect_rep
```

```
[1] "R"  "R"  "R"  "ok"
```

Vector type

The type of data the vector is made of.

```
1 class(num_vect)
```

```
[1] "numeric"
```

```
1 class(char_vect)
```

```
[1] "character"
```

Vector type

A vector must be made of **elements all of the same type**!

```r
wrong <- c(1,2,3,"don't know", 4)
class(wrong)
```

```
[1] "character"
```

```r
wrong
```

```
[1] "1"          "2"          "3"          "don't know" "4"
```

Otherwise you “risk” everything being converted to character.

```r
correct <- c(1,2,3,NA, 4)
class(correct)
```

```
[1] "numeric"
```

```r
correct
```

```
[1]  1  2  3 NA  4
```

is.* & as.*

We can test or convert (when possible) the type of a vector with the **`is.`** & **`as.`** functions

A `character` vector

```
1 char_vect
```

```
[1] "R"  "R"  "R"  "ok"
```

```
1 is.character(char_vect)
```

```
[1] TRUE
```

```
1 as.numeric(char_vect) #!!
```

```
[1] NA NA NA NA
```

is.* & as.*

A `numeric` vector

```
num_vect
```

```
[1] 1 2 3 4
```

```
is.numeric(num_vect)
```

```
[1] TRUE
```

```
as.character(num_vect) #!!
```

```
[1] "1" "2" "3" "4"
```

is.* & as.*

A `logical` vector

```
logi_vect <- c(TRUE,FALSE,TRUE)
is.logical(logi_vect)
```

```
[1] TRUE
```

```
as.character(logi_vect)
```

```
[1] "TRUE"  "FALSE" "TRUE"
```

```
is.numeric(logi_vect)
```

```
[1] FALSE
```

```
as.numeric(logi_vect)
```

```
[1] 1 0 1
```

Vector attributes

Each element of a vector can be given a name.

```
1 num_vect
```

```
[1] 1 2 3 4
```

```
1 names(num_vect) # no names assigned
```

```
NULL
```

```
1 letters[1:4]
```

```
[1] "a" "b" "c" "d"
```

```
1 names(num_vect) <- letters[1:4]
2
3 num_vect
```

```
a b c d
1 2 3 4
```

Every object is characterized by a dimension (**dim()**); however, since a vector is one-dimensional, we use the **length()** command to get the length of the vector

```
1 num_vect
```

```
a b c d
1 2 3 4
```

```
1 dim(num_vect)
```

```
NULL
```

```
1 length(num_vect)
```

```
[1] 4
```

Indexing

We can select, remove and extract elements simply by using the position index inside square brackets vector[pos] .

```
# Create a vector of 10 random numbers drawn between 1 and
my_vect <- round(runif(n = 10,min = 1, max = 100))
my_vect
```

```
[1] 47 42 91 15 74 98 88 13 55 15
```

```
my_vect[1] # extract the first element
```

```
[1] 47
```

Indexing

We can select, remove and extract elements simply by using the position index inside square brackets vector[pos] .

```
1 my_vect[1:5] # extract the first 5 elements
```

```
[1] 47 42 91 15 74
```

```
1 my_vect[c(1,4,2,9)] # extract elements of our choice
```

```
[1] 47 15 42 55
```

```
1 my_vect[length(my_vect)] # last element (why?)
```

```
[1] 15
```

Negative Indexing

In the same way, we can extract all the elements of the vector except some

```
my_vect[-c(1)] # all except the first element
```

```
[1] 42 91 15 74 98 88 13 55 15
```

```
my_vect[-c(1:9)] # all except the first 9
```

```
[1] 15
```

If we assign names to the elements of the vector, we can index them by “calling them by name” (less common)

```
letters[1:length(my_vect)]
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```
# assign names
names(my_vect) <- letters[1:length(my_vect)]
```

```
names(my_vect) # What are the names of the elements of my_
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```
my_vect
```

```
 a  b  c  d  e  f  g  h  i  j 
47 42 91 15 74 98 88 13 55 15 
```

```
my_vect["a"] # I want the element of my_vect called "a"
```

```
 a 
47 
```

Logical Indexing

We can select elements of a vector based on specific logical conditions: **TRUE** and **FALSE**. The idea is that if we have a vector of length **n** and another logical vector of length **n**, all the **TRUE** elements will be selected:

```
numbers <- 1:7
numbers
```

```
[1] 1 2 3 4 5 6 7
```

```
numbers>2 & numbers<5
```

```
[1] FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE
```

```
numbers[numbers>2 & numbers<5]
```

```
[1] 3 4
```

```
as.numeric(numbers>2 & numbers<5)
```

```
[1] 0 0 1 1 0 0 0
```

It doesn't only work for the elements of the numbers vector, but for any vector!

```
1 my_letters <- letters[1:7]
2 my_letters
```

```
[1] "a" "b" "c" "d" "e" "f" "g"
```

```
1 numbers>2 & numbers<5
```

```
[1] FALSE FALSE  TRUE  TRUE FALSE FALSE FALSE
```

```
1 my_letters[numbers>2 & numbers<5]
```

```
[1] "c" "d"
```

Positional Indexing

With the `which()` function we can get the positions associated with a logical selection:

```
1 my_letters
```

```
[1] "a" "b" "c" "d" "e" "f" "g"
```

```
1 which(my_letters == "a")
```

```
[1] 1
```

Positional Indexing

Example:

```
1 # create a vector of 10 random integers between 1 and 40
2 my_vect <- round(runif(10, min = 1, max = 40))
3 my_vect
```

```
 [1]  1 30 30 26 35  7 37 23 36 31
```

What is the difference between these 2 commands?

```
1 which(my_vect > 20)
```

```
[1]  2  3  4  5  7  8  9 10
```

```
1 my_vect[which(my_vect > 20)]
```

```
[1] 30 30 26 35 37 23 36 31
```

Indexing and Assignment

Indexing can also be useful to assign a value to a given element of the vector:

```
1 my_letters
```

```
[1] "a" "b" "c" "d" "e" "f" "g"
```

```
1 my_letters[1] <- "z"
2 my_letters
```

```
[1] "z" "b" "c" "d" "e" "f" "g"
```

```
1 my_letters[9] <- "i"
2 my_letters
```

```
[1] "z" "b" "c" "d" "e" "f" "g" NA  "i"
```

Indexing and Assignment

I replace the NA value with the missing letter

```
1 my_letters
```

```
[1] "z" "b" "c" "d" "e" "f" "g" NA  "i"
```

```
1 is.na(my_letters) # are there elements of my_letters = NA?
```

```
[1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE  TRUE FALSE
```

```
1 which(is.na(my_letters)) # in which positions are these el
```

```
[1] 8
```

Indexing and Assignment

I replace the NA value with the missing letter

```
1 # indeed, if I look at the indicated position
2 my_letters[is.na(my_letters)]
```

```
[1] NA
```

```
1 my_letters[which(is.na(my_letters))]  # equivalent
```

```
[1] NA
```

```
1 my_letters[is.na(my_letters)] <- "h" # replace NA with the
2 my_letters
```

```
[1] "z" "b" "c" "d" "e" "f" "g" "h" "i"
```

Mathematical operations on vectors

Mathematical operations on vectors

We can perform operations on vectors, applying the same operation to all the elements of the vector (element-wise)

```
1 new_vect <- rep(2:4, each = 2)
2 new_vect
```

```
[1] 2 2 3 3 4 4
```

```
1 # you can perform any operation
2 new_vect/2
```

```
[1] 1.0 1.0 1.5 1.5 2.0 2.0
```

```
1 log(new_vect)
```

```
[1] 0.6931472 0.6931472 1.0986123 1.0986123 1.3862944
1.3862944
```

Summary Statistics

The most common operations are, for example, the mean `mean()`, the standard deviation `sd()`, the median `median()`, the maximum `max()` and minimum `min()` values.

```
# create a vector by sampling 1000 values from a normal di
new_vect <- rnorm(n = 1000, mean = 1, sd = 4)
mean(new_vect)
```

```
[1] 0.8745842
```

```
sd(new_vect)
```

```
[1] 3.940867
```

```
median(new_vect)
```

```
[1] 0.8438001
```

Summary Statistics

```r
summary(new_vect)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max. 
-10.7889  -1.6843   0.8438   0.8746   3.5015  12.8939 
```

You can also use the `describe` function from the `psych` package:

```r
psych::describe(new_vect)
```

```
   vars    n mean   sd median trimmed  mad    min   max range
skew kurtosis
X1    1 1000 0.87 3.94   0.84    0.88 3.87 -10.79 12.89 23.68
0.01    -0.15
     se
X1 0.12
```

NA values

Sometimes (very often) we may have missing data

```
new_vect[1] <- NA # Assign NA to the first element of the

mean(new_vect) # the mean cannot be computed!
```

```
[1] NA
```

```
mean(new_vect, na.rm = TRUE) # solution!
```

```
[1] 0.8783685
```

Frequencies

Another common operation performed on vectors (and on other data structures) is counting frequencies

```
1 # let's go back to the vector created at the beginning of
2 char_vect
```

```
[1] "R"  "R"  "R"  "ok"
```

```
1 # with the table function we can compute frequencies
2 table(char_vect)
```

```
char_vect
ok  R
 1  3
```

Let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR